

Qualcomm[®] Hexagon[™] aDSP and cDSP Stability Debugging Guide

80-N4597-2 Rev. B

About Qualcomm

Qualcomm relentlessly innovates to deliver intelligent computing everywhere, helping the world tackle some of its most important challenges. Building on our 40 years of technology leadership in creating era-defining breakthroughs, we deliver a broad portfolio of solutions built with our leading-edge AI, high-performance, low-power computing, and unrivaled connectivity. Our Snapdragon[®] platforms power extraordinary consumer experiences, and our Qualcomm Dragonwing[™] products empower businesses and industries to scale to new heights. Together with our ecosystem partners, we enable next-generation digital transformation to enrich lives, improve businesses, and advance societies. At Qualcomm, we are engineering human progress.

Confidential – Qualcomm Technologies, Inc. and/or its affiliated companies – May Contain Trade Secrets

NO PUBLIC DISCLOSURE PERMITTED: Please report postings of this document on public servers or websites to: DocCtrlAgent@qualcomm.com.

© Qualcomm Technologies, Inc. and/or its subsidiaries. All rights reserved.

Revision History

Revision	Date	Description
B	February 2026	Numerous changes have been made to this document; it must be read in entirety.
A	March 2017	Initial release

Qualcomm
Confidential - May Contain Trade Secrets
2026-05-12 06:30:20 GMT
hyungchul.won@lge.com

Contents

1 Hexagon DSP Programming Model	<u>4</u>
2 Code Organization	<u>8</u>
3 Debug Environment	<u>11</u>
4 Collect RAM Dumps	<u>13</u>
5 Debugging Using Crashman	<u>19</u>
6 Case Studies	<u>42</u>
7 References	<u>57</u>

Qualcomm
Confidential - May Contain Trade Secrets
2026-05-12 06:30:20 GMT
hyungchul.won@lge.com



Section 1

Hexagon DSP Programming Model

Qualcomm
Confidential - May Contain Trade Secrets
2026-05-12 06:30:20 GMT
hyunchul.won@lge.com

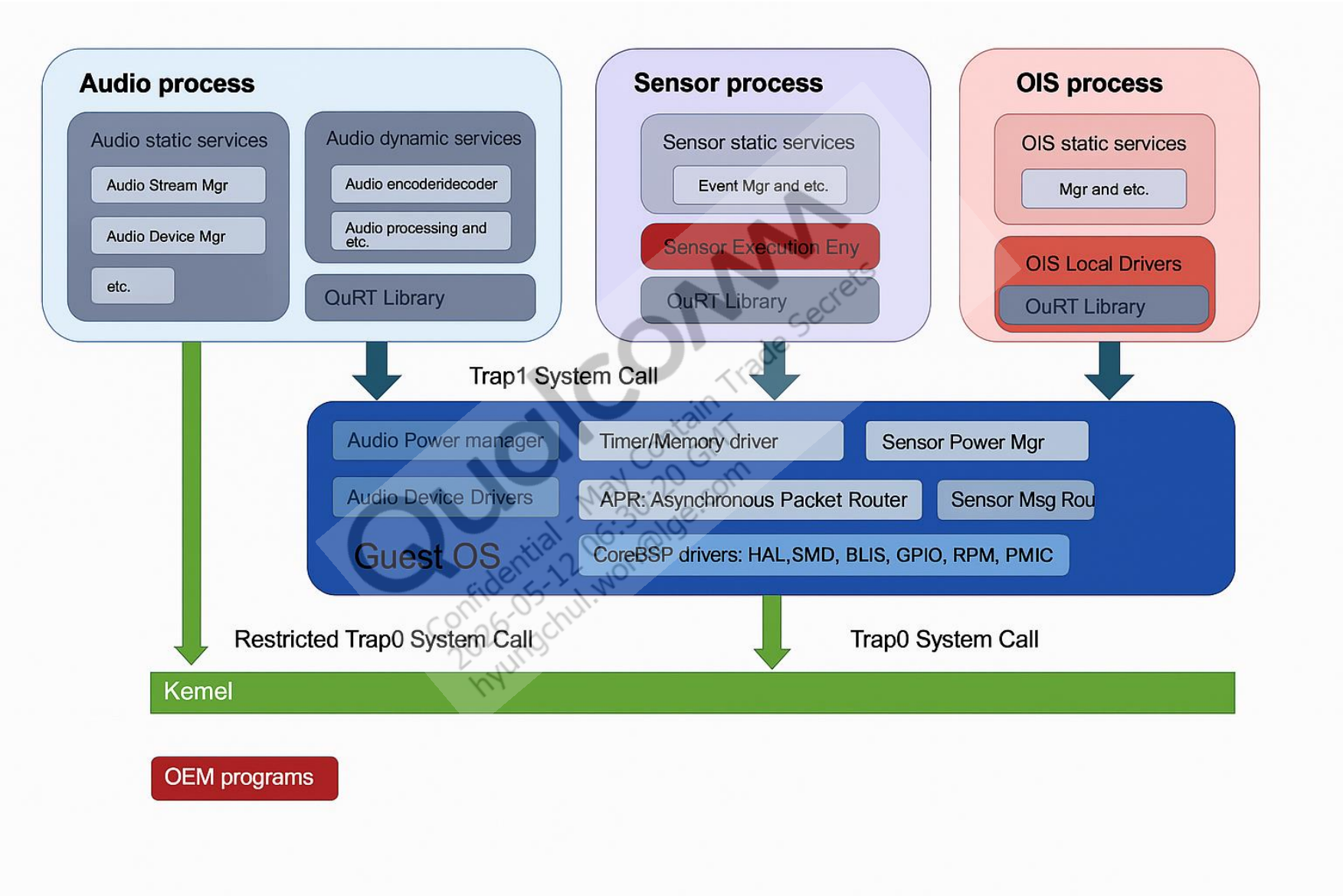
aDSP and cDSP Architecture

- NOTE : This whole document is based on SM8850 target and paths may vary for other targets.
- Qualcomm® Hexagon™ is a multi-threaded processor. The SM8850 device integrates advanced Hexagon cores with enhanced concurrency and compute capabilities.
- **Audio DSP (aDSP)**
 - Handles audio, voice, and legacy DSP processing.
 - Based on Hexagon V81 architecture.
 - Includes 8 hardware threads where each thread:
 - Has 32 general and control registers, each 32-bit wide.
- **Compute DSP (cDSP)**
 - Handles compute workloads dedicatedly using Qualcomm® Hexagon™ Vector eXtensions (HVX).
 - Includes 8 hardware threads where each thread:
 - Has 32 general and control registers, each 32 bits wide.
 - Supports vector registers of 512 bits through HVX units.
 - Integrates multiple HVX coprocessors for high-performance compute.

Protection Domains

- aDSP is a real-time processor and its stability critically affects the overall user experience.
- Different protection domains (PDs) exist in the aDSP software architecture for SM8850 to ensure kernel stability and safeguard QTI's proprietary hardware information.
- Following are the three protection domains in the aDSP:
 - **Kernel mode**
 - Access all system registers and all physical memory
 - Kernel runs in kernel mode: non-preemptible with small memory foot-print
 - **Guest OS mode**
 - Access limited system registers memory mapped with OS or User mode
 - QTI drivers, most RTOS services such as memory, process management
 - **User mode**
 - Runs OEM programs, user space drivers, RTOS user libraries

Protection Domains





Section 2

Code Organization

Qualcomm
Confidential - May Contain Trade Secrets
2026-05-12 06:30:20 GMT
hyunchul.won@lge.com

aDSP Code Organization

- aDSP build names are in the format – LPAIDSP.HT.1.2-00749.1-KAANAPALI-1
- `adsp_proc\build` contains the build directory
 - `adsp_proc\build\ms\dynamic_modules\kaanapali.adsp.prod` contains generated .so files
 - `adsp_proc\build\ms` contains the elf files
 - `UKERNEL_kaanapali.adsp.prodQ.elf`
 - `ROOT_kaanapali.adsp.prodQ.elf`
 - `AUDIO_kaanapali.adsp.prodQ.elf`
 - `SENSOR_molokai.adsp.prodQ.elf`
 - `OIS_kaanapali.adsp.prodQ.elf`
 - `adsp_proc\crashman` contains the crashman script files
 - `adsp_proc\qdsp6\Sysmon` contains the sysmon profile tool

cDSP Code Organization

- cDSP build names are in the format – `CDSP.HT.3.2-00673.1-KAANAPALI-1`
- `cdsp_proc\build` contains the build directory
 - `cdsp_proc\build\ms\dynamic_modules\kaanapali.cdsp.prod` contains the `.so` files
 - `cdsp_proc\build\ms` – contains the `elf` files
 - `UKERNEL_kaanapali.cdsp.prodQ.elf`
 - `ROOT_kaanapali.cdsp.prodQ.elf`
 - `cdsp_proc\crashman` – contains the crashman script files
 - `cdsp_proc\qdsp6\sysmon` – contains the sysmon profile tool

Qualcomm
Confidential - May Contain Trade Secrets
2026-05-12 06:30:20 GMT
hyungchul.won@lge.com



Section 3

Debug Environment

Qualcomm
Confidential - May Contain Trade Secrets
2026-05-12 06:30:20 GMT
hyunchul.won@lge.com

Tools Required

- Python: Python 3.x or Latest version
- QXDM: v5 or latest from QPM
- adb and fastboot: Latest available versions
- TRACE32:
 - Download TRACE32 that supports Hexagon v72 or later versions
 - Copy the TRACE32 installer to the c:\T32 folder
- aDSP and cDSP Crashman: Available in aDSP or cDSP builds located at:
 - cdsp_proc\crashman
 - adsp_proc\crashman
- xPCAT: Latest from QPM

Qualcomm
Confidential - May Contain Trade Secrets
2026-05-12 06:30:20 GMT
hyungchul.won@lge.com



Section 4

Collect RAM Dumps

Qualcomm
Confidential - May Contain Trade Secrets
2026-05-12 06:30:20 GMT
hyunchul.won@lge.com

Disable Sub-system Restart

- Sub-system restart (SSR) is enabled to avoid device entering the download mode during a system crash
- To disable SSR on aDSP or cDSP using remoteproc instance:
 - Step 1: Run the following commands:

```
adb root
adb shell
cat /sys/class/remoteproc/remoteproc*/name
/*check if the name is adsp or cdsp, if not look for all other devices remoteproc1, remoteproc2,
remoteproc2....
look for name 'adsp' to disable SSR on adsp
look for name 'cdsp' to disable SSR on cdsp */
```
 - Step 2: Run the following command if remoteproc0 instances has aDSP or cDSP name attribute:

```
echo "disabled" > /sys/class/remoteproc/remoteproc0/recovery
```
- To disable SSR using system properties.
 - Run the following command:

```
adb shell setprop persist.vendor.ssr.restart_level ALL_DISABLE
```
 - Note that this command disables SSR for all subsystems including aDSP and cDSP.

Collect RAM Dump by Inducing Crash

- Enter any of the following commands in the Command tab of the QXDM Professional™ Tool to trigger a crash on aDSP:

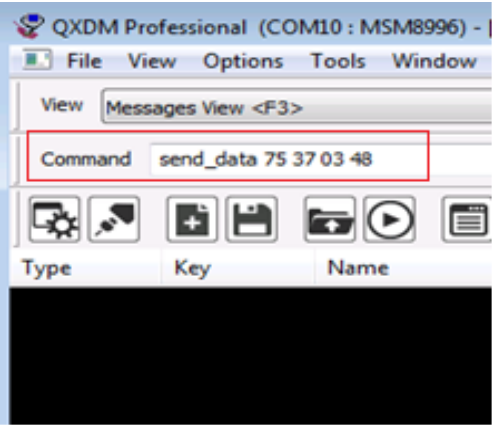
QXDM command	Error type
send_data 75 37 03 48	Software error fatal
send_data 75 37 03 48 03	Software exception (Div by 0)
send_data 75 37 03 48 02	Software exception (NULL ptr)

- Enter any of the following commands in the Command tab of the QXDM Professional tool to trigger a crash on cDSP

QXDM command	Error type
send_data 75 37 03 96	Software error fatal
send_data 75 37 03 96 03	Software exception (Div by 0)
send_data 75 37 03 96 02	Software exception (NULL ptr)

- Android Kernel Panic is used for both aDSP and cDSP

```
adb root
adb shell
echo c > /proc/sysrq-trigger
```



RAM Dump Files

Collect the following files for RAM files to parse the collected dumps:

- DSP elf files listed as follows

aDSP (adsp_proc/build/ms)	cDSP(cdsp_proc/build/ms)
UKERNEL_kaanapali.adsp.prodQ.elf	UKERNEL_kaanapali.cdsp.prodQ.elf
ROOT_kaanapali.adsp.prodQ.elf	ROOT_kaanapali.cdsp.prodQ.elf
AUDIO_kaanapali.adsp.prodQ.elf	
SENSOR_molokai.adsp.prodQ.elf	
OIS_kaanapali.adsp.prodQ.elf	

- Dump files which are located in QPST folder at C:\ProgramData\Qualcomm\QPST\Sahara\<crashing COM Port>. These files can be stored at a location of your choice.
- vmlinux file located at the following location
 <android_build>/out/target/product/<chipset>/obj/KERNEL_OBJ
- Shared object files found at the following location if the crash is in any shared object:
 - adsp_proc\build\dynamic_signed\adsp.prod\shared_obj
 - cdsp_proc\build\dynamic_signed\cdsp.prod\shared_obj

Enable ETM on cDSP

- The Embedded trace macro (ETM) triggers the ISDB breakpoints.
- To enable ETM traces run following adb commands for 32 KB buffer size:

```
sh /vendor/bin/coresight_reset_source_sink.sh
echo "reset_source_sink executed"
echo 1 > /sys/bus/coresight/devices/tmc_etr0/enable_sink
echo 0x10000000 > /sys/bus/coresight/devices/tmc_etr0/buffer_size
/vendor/bin/qdss_qmi_helper cdsp etm_config "dsp testbus 0x80FF0010"
/vendor/bin/qdss_qmi_helper cdsp etm_config "dsp mode 0x03"
echo 1 > /sys/bus/coresight/devices/turing_etm0/enable_source
```

- ETM trace are collected enabling the device crash.
- Share dumps with QTI for further analysis.

Enable ETM on aDSP

- The ETM triggers the ISDB breakpoints.
- To enable ETM traces run following adb commands:

```
sh /vendor/bin/coresight_reset_source_sink.sh
echo "reset_source_sink executed"
echo 1 > /sys/bus/coresight/devices/tmc_etr0/enable_sink
echo 0x10000000 > /sys/bus/coresight/devices/tmc_etr0/buffer_size
/vendor/bin/qdss_qmi_helper adsp etm_config "dsp testbus 0x80FF0010"
/vendor/bin/qdss_qmi_helper adsp etm_config "dsp mode 0x03"
/vendor/bin/qdss_qmi_helper adsp etm_config "dsp mode 0x05"
echo mem > /sys/bus/coresight/devices/coresight-tmc-etr/out_mode
echo 0x2000000 > /sys/bus/coresight/devices/coresight-tmc-etr/buffer_size
echo 1 > /sys/bus/coresight/devices/coresight-tmc-etr/enable_sink
echo 1 > /sys/bus/coresight/devices/coresight-audio-etm0/enable_source
```

- ETM trace are collected enabling the device crash.
- Share dumps with QTI for further analysis.



Section 5

Debugging Using Crashman

Qualcomm
Confidential - May Contain Trade Secrets
2026-05-12 06:30:20 GMT
hyunchul.won@lge.com

Generate Crashman Report

- The procedure to generate crashman report is same for aDSP and cDSP.
- Enter aDSP build for aDSP crashman report and cDSP build for cDSP crashman report.
- To generate crashman report:
 - Get aDSP or cDSP build location.
 - pushd the build location for aDSP or cDSP:
 - aDSP: `pushd <adsp_build>\adsp_proc\qdsp6\scripts\crashman`
 - cDSP: `pushd <cdsp_build>\cdsp_proc\qdsp6\scripts\crashman`
 - Run the following commands:

```
Python adspcrashman.py
-t <designated target>
-b <location of DSP build>
-o <output location>
-d <location of crash dump>
-e <adsp/cdsp elf files location>
-smmu64 <location of vmlinux>
-sofile <location of shared object files>
-ss cdsp
```
 - Ensure to pass the `-ss cdsp` argument when parsing cDSP crash dumps. If this argument is not specified, the script parses aDSP crash dumps by default.

Generate Crashman Report (cont.)

■ Example for crashman commands are as follows:

□ aDSP

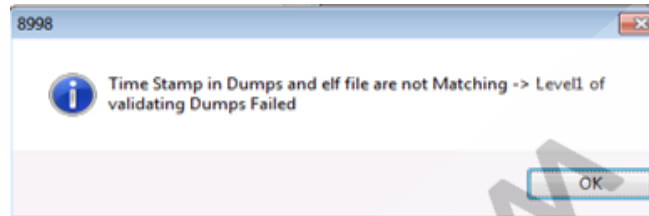
```
Python adspcrashman.py
-t sdm660
-d c:\Port_COM148\DDRCS0.BIN
-o c:\temp\
-b <adsp_build>\adsp_proc
-e adsp_proc\build\ms
-smmu64 c:\vmlinux_elf_path\
-sofile adsp_proc\hap\oem\build\dynamic_modules_hash
```

□ cDSP

```
Python adspcrashman.py
-t sdm660
-d c:\Port_COM148\DDRCS0.BIN
-o c:\temp\
-b <cdsp_build>\cdsp_proc
-e adsp_proc\build\ms
-smmu64 c:\vmlinux_elf_path\
-sofile cdsp_proc\hap\oem\build\dynamic_modules_hash
```

Check Timestamp

- If the timestamp of the dump is not matching with `elf`, crashman shows a popup message due to timestamp mismatch:



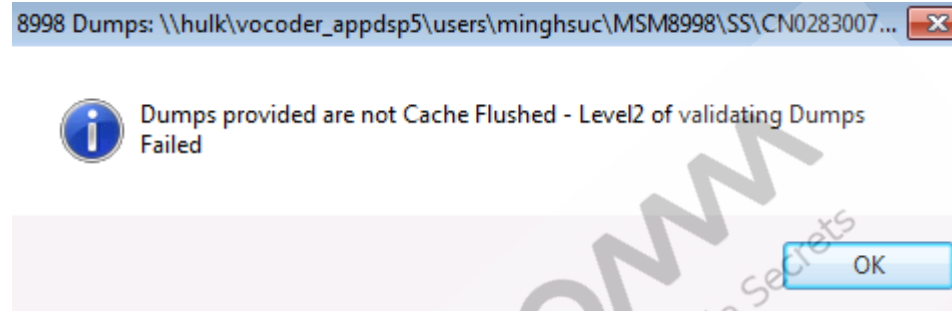
- Check timestamp of dump and `elf` as shown in the following images. To generate all symbol files the timestamp of dumps and `elfs` timestamp must match. For more information, see [Collect RAM Dump by Inducing Crash](#).

```
C:\>strings \Port_COM29\DDRCSS0_0.BIN | grep "Q6_BUILD"
ENG time:Q6_BUILD_TS_Sat_Feb_25_02:08:08_PST_2017_sdm660_qualcomm.com
QCOM time:Q6_BUILD_TS_Sat_Feb_25_02:08:08_PST_2017_ADSP.VT.4.0-00214-SDM660
ENG time:Q6_BUILD_TS_Sat_Feb_25_02:08:08_PST_2017_sdm660_qualcomm.com
QCOM time:Q6_BUILD_TS_Sat_Feb_25_02:08:08_PST_2017_ADSP.VT.4.0-00214-SDM660
```

```
C:\>strings \\adsp_proc\build\ms\AUDIO_660.adsp.prodQ.elf | grep "Q6_BUILD"
ENG time:Q6_BUILD_TS_Fri_Feb_24_04:50:34_PST_2017_sdm660_qualcomm.com
QCOM time:Q6_BUILD_TS_Fri_Feb_24_04:50:34_PST_2017_CDSP.VT.1.0-00129-SDM660
```

Check Cache Flush Status

- Dumps provided must have cache flushed, else a pop-up message appears as follows:



- If you see the above message, then:
 - Run the dump collection procedure. For more information, see [Collect RAM Dump by Inducing Crash](#).
 - Reproduce the issue and check if cache is flushed or persists.
 - Raise a Salesforce case and report the issue to QTI if you still see the issue.

Crashman Output

- After successful dump loading on T32, crashman report generates `dspanalysis.txt` file similar to the following image:

```
Cache Status
=====
Cache Flush Status      : The DUMPS provided are Cache Flushed

QURT System State Status
=====
QURT_system_state = 36 ==> QURT_LOAD_NOTIFIER_TCB

Running CallStack
=====
-000|Clock_SetCPUMaxFrequencyAtCurrentVoltage()
-001|DalClock_SetCPUMaxFrequencyAtCurrentVoltage()
-001|sleepOS_setFrequency()
-001|sleepOS_performLPMEnter()
-001|sleepOS_performLPM()
-002|sleepOS_mainTask()
-003|qurt_trampoline()
----|end of frame

Stack Walker CallStack
=====
Clock_SetCPUMaxFrequencyAtCurrentVoltage (0xF01D77EC)
sleepOS_performLPM (0xFFFFFFFFF01A91C4)
sleepOS_performLPM (0xF01A91C4)
sleepOS_mainTask (0xF0015BE8)
qurt_trampoline (0xF0475890)

Thread Name
=====
Crashed SW Thread      : sleep
Crashed PD             : GuestOS

Cause For ADSP Crash :
Exception detected in thread with TCB:0xF00069C0
TLB Miss Read occurred at address - 0x0, pc - 0xF01D77EC
Cause For ADSP Crash :
SW error, fatal exit error

Qurt Error Info
=====
SW error, fatal exit error
```

Shows cache flush status

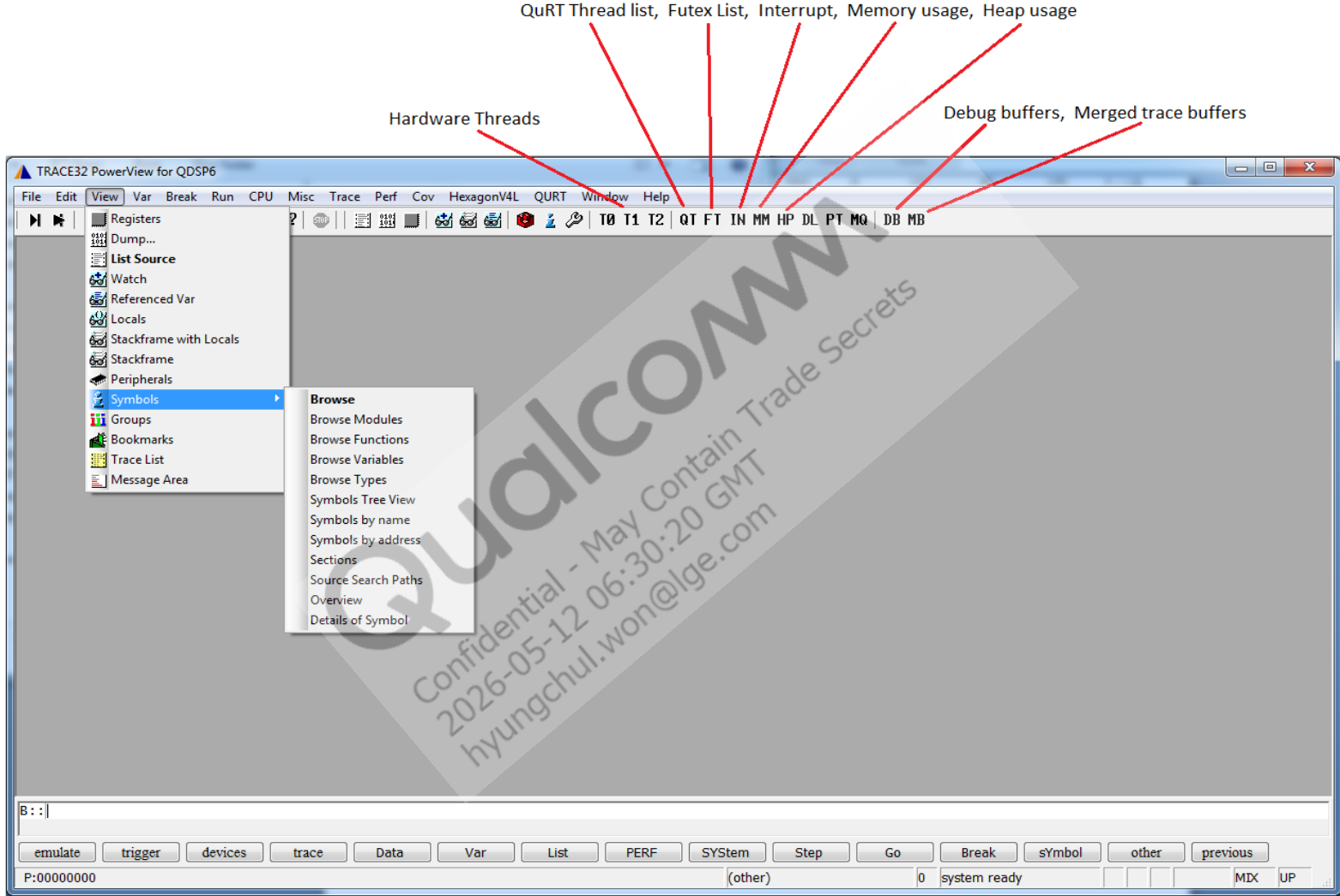
Shows detailed call stack of process where exception happened

Shows call stack of process where exception happened

Shows software thread name where exception occurred

Details on what error occurred inside the software thread

Understanding TRACE32 Commands

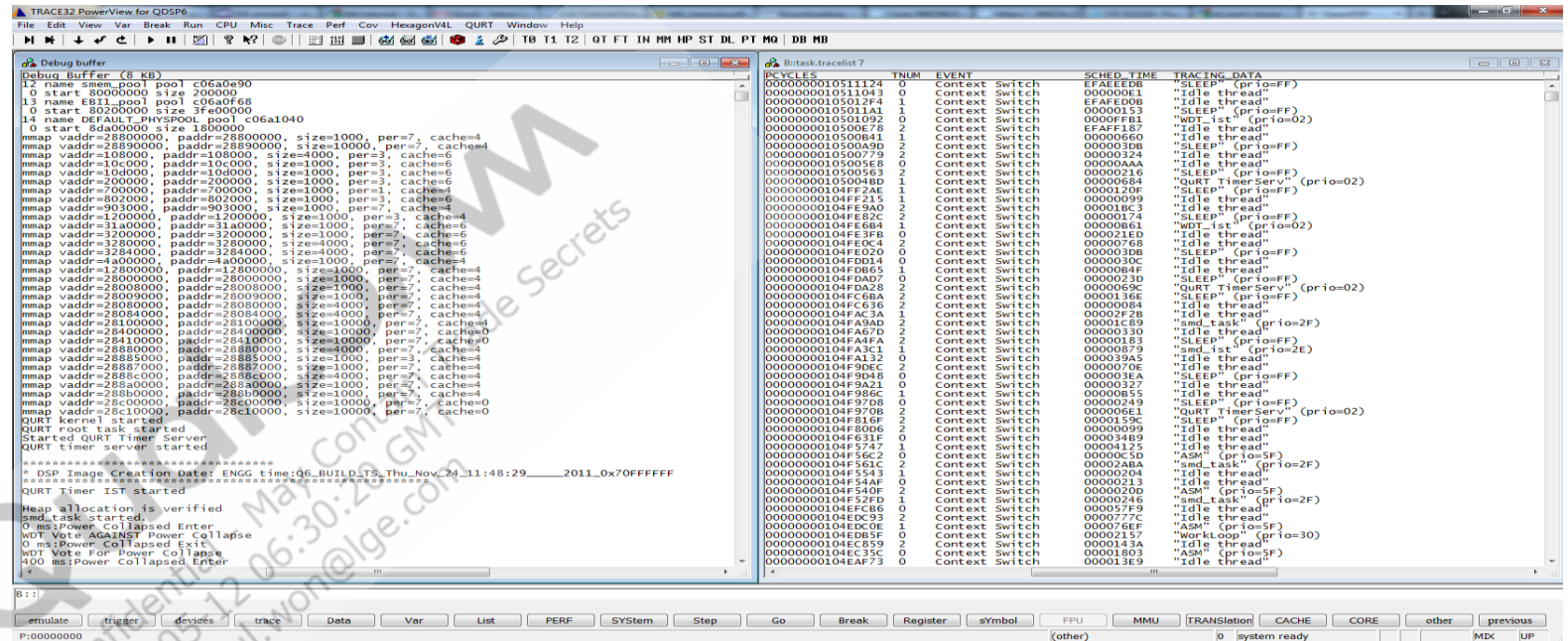


Understanding TRACE32 Commands (cont.)

- **T0, T1, T2, and T3** are the hardware threads. Type 'r' to display the registers.
- Select the **T0** button to display the hardware registers for **T0** threads. Similarly, for hardware registers of other threads, select the corresponding button.
- **QT** is the QuRT™ software thread button and displays the QuRT threads at the time the dump is generated.
- **FT** button is the Futex list and displays all the engaged futexes and displays deadlocks, if any.
- **IN** displays the interrupt list.
- **MM** lists the free Memory blocks available.
- **HP** gives the heap usage while the dump is being extracted.
- **DB** shows the debug buffers that are stored in the debug buffer. For example, QuRT debug messages specifically `printf` messages.
- **MB** displays and merges:
 - Hardware threads
 - Trace buffers. Trace buffers provide information about:
 - QuRT threads running on a particular hardware thread
 - Context switches

Understanding TRACE32 Commands (cont.)

- The window on the left is the **Debug buffers (DB)** and on the right is **Merge buffer (MB)**.
- This window shows all the QuRT related printf messages. The list of main messages displayed are:
 - Heap initialization
 - Running for the QuRT
 - Power collapse enter and
 - Power collapse exit
 - Reason for the power collapse exit and time of power collapse enter.
- **MB** window displays all the hardware thread with their own associated Trace buffers. These Trace buffers can be accessed by selecting **QuRT-Trace Buffers** from the main menu.
- The **MB** displays all hardware buffers merged into one.



Qurt_error_info

- `Qurt_error_info` is a global variable useful for categorizing the type of exception.
- `analy_qurt_error.cmm` in the Crashman tool automatically interprets `Qurt_error_info` with the following result:

```
Checking QURT_error_info structure
Exception list
  TCB      HWTID  errorcode  pcycle
0xF07A5C40 0x2    0x0      0x9B120E49C
Loading CPU [0x0] registers
-----
SW error, fatal exit error
-----
error TCB 0xF07ADBC0
Loading CPU [0x1] registers
Loading CPU [0x2] registers
Done
```

Qurt_error_info breakdown

- v.v QURT_error_info indicates the Qurt_error_info breakdown:

QuRT Error code – 12 – ERR_FATAL_EXIT

```
B::v.v QURT_error_info
QURT_error_info = (
  status = (
    hw_tnum = 0,
    rsvd1 = 0,
    cause2 = 0,
    cause = 12,
    rsvd2 = 0,
    ext_nmi_hw2 = 0,
    ext_nmi_hw1 = 0,
    ext_nmi_hw0 = 0,
    local_nmi = 1,
    fatal = 1,
    status = 1,
    lock = 0),
  error_param = 0,
  globregs = (syscfg = 8627327, modect1 = 393223),
  locregs = ((r0 = 3217014719, r1 = 3217014719, ...),
  users_errors = (
    entry = (
      error_tcb = 0xF07A5C40,
      error_code = 0,
      hw_thread = 2,
      pcycle = 0x00000009B120E49C),
      (error_tcb = 0x0, error_code = 0, hw_thread = ...),
      (error_tcb = 0x0, error_code = 0, hw_thread = ...),
      (error_tcb = 0x0, error_code = 0, hw_thread = ...),
      counter = 1))
```

This is set to 1 if NMI is triggered externally (LPASS watchdog hardware)

This is set to 1 if NMI is triggered internally by QuRT

Error thread

QURT thr

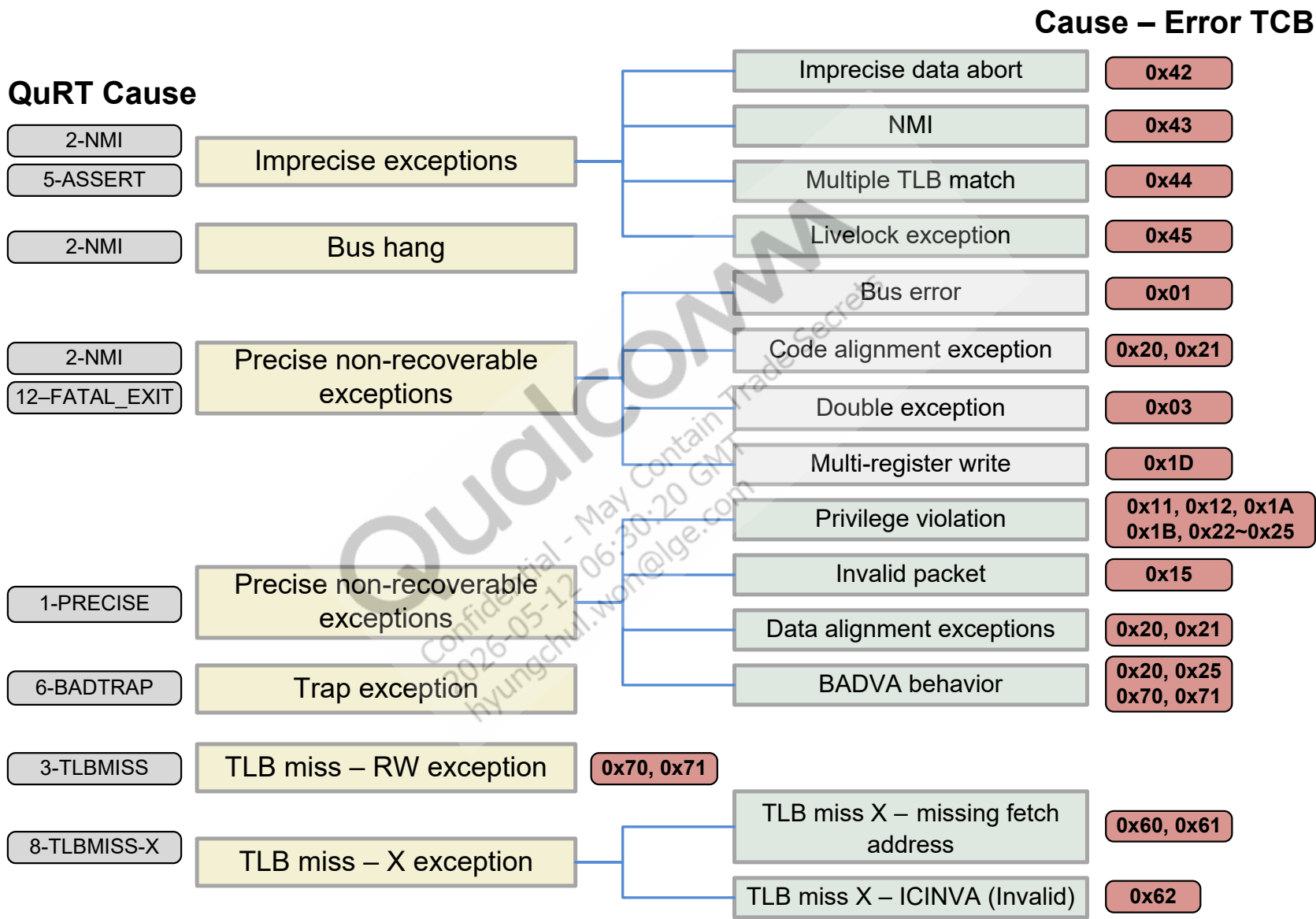
tcbptr
F07A5B00
F07A5C40
F07A5D80
F07A5EC0
F07A6000
F07A6500
F07A6640
F07A6780
F07A68C0
F07A6A00
F07A6B40
F07A6C80
F07A6DC0
F07A6F00
F07A7040
F07A7180
F07A72C0
F07A7400
F07A7540
F07A7680
F07A77C0
F07A7900
F07A7A40
F07A7B80

QuRT Error Types

■ QuRT handles the following types of exceptions:

- Application exceptions (fatal error)
- Kernel exceptions:
 - Raised by the QuRT kernel
 - Include the following:
 - Supervisor-mode exceptions
 - Page faults
 - Hexagon processor exceptions
- Imprecise exceptions:
 - Displays serious and unrecoverable error conditions that can be raised in either the QuRT kernel or the application code
 - Stores bad addresses, hardware parity errors, and other imprecise slave error conditions
 - Includes NMI exceptions raised outside the Hexagon processor

Exceptions



Imprecise Exceptions

- Serious and unrecoverable error conditions
- Four possible sources:
 - Imprecise data abort:
 - Triggered by a store operation that results in a fatal error from the slave device. For example: a bad address, hardware parity error, or other imprecise slave error condition
 - These errors are serious and unrecoverable conditions
 - NMI pin is asserted, that is, another process or subsystem crash can trigger NMI and shut down the DSP
 - Multiple translation lookaside buffer () matches:
 - must not be programmed with multiple matching entries; otherwise, its behavior is undefined
 - If a hardware lookup matches multiple entries, an imprecise exception may be raised to help OS developers debug this type of error
 - This exception may arise due to issues in implementation procedure or the conditions during the implementation
 - Livelock exception:
 - Hardware can detect if a thread is not making sufficient progress, and raise a livelock exception
 - May indicate a hardware problem

Precise Nonrecoverable Exceptions

- When a precise exception occurs, the exception point is established before the packet that caused the exception
- The following exceptions may not be recoverable:
 - BUS error:
 - Observed when a blocking memory instruction makes a request to the system resulting in an error
 - Typically the result of a bad address
 - Code alignment exception:
 - Observed when a change-of-flow instruction is executed to an address that is not word-aligned
 - For example, when executing an indirect JUMPR instruction with a bad pointer
 - Double exception:
 - Observed when any exception event happens while the thread is in an exception mode
 - Upon taking a double exception, the hardware diag is filled with the cause code of the event that triggered the double exception
 - For details on cause codes, see *Hexagon V5 Architecture System-Level Architecture* (80-V9418-16)
 - Multiregister write: Precise exception cause 0x1D occurs for a packet with multiple writes to the same destination register

Precise Recoverable Exceptions

■ Privilege violations

- Detected when the memory management unit (MMU) is enabled and a memory page is accessed through either a fetch or load/store
 - In its current operating mode, the thread does not have permission to access that memory page
- Raised when a violating thread is not allowed to access

■ Invalid packet

- Caused by a packet that places non-zero bits into a reserved encoding areas

■ Data alignment exceptions

- All the data loads and stores must be naturally aligned to their access size
- For example, a data alignment exception is raised when a MEMW word load is not word-aligned (the lowest 2 bits of the effective address are nonzero)

Precise Recoverable Exceptions (cont.)

■ BADVA behavior

- Data-side exceptions that can change the underlying registers (BADVA0, BADVA1, BVS, V0, and V1) that make up BADVA's read-only contents include:
 - Precise error exceptions with causes 0x20 through 0x25
 - missed RW exceptions with causes 0x70 and 0x71
- Following slides describe the BADVA underlying register changes for these exceptions

Qualcomm
Confidential - May Contain Trade Secrets
2026-05-12 06:30:20 GMT
hyunchul.won@lge.com

Miss-XRW Exception

- Generated either when the required address translation for a fetch is not found in the , or when the address referred to is an instruction cache invalid virtual address (ICINVA)
- without an address may be observed:
 - miss-X is due to a missing fetch address
 - Occurs when a miss-X exception originated from a fetch that was missed in the
 - Software at the miss handler is responsible for refilling the with the needed translation
 - Upon returning from the exception, the packet that caused the miss restarts
 - miss-X is due to an ICINVA
 - -miss-X exception behavior is different for an exception due to an ICINVA instruction
 - Raised when the invalid address is not present in the
 - Faulting page is put into BADVA, and the cause code is set to 0x2
 - Occurs when a miss-X is due to an ICINVA where the invalidated address is not mapped in the

Miss-XRW Exception (cont.)

- miss-RW exception
 - Generated when a load/store virtual address cannot be found
 - Software at the miss handler is responsible for refilling the with the needed translation
 - Upon returning from the exception, the packet that caused the miss is restarted

Qualcomm
Confidential - May Contain Trade Secrets
2026-05-12 06:30:20 GMT
hyunchul.won@lge.com

TRAP Exception

- Caused when the software `TRAP0` or `TRAP1` instruction is executed
 - Trap instruction takes an 8-bit unsigned immediate value as an operand
- This immediate field is placed in the cause field of SSR
- TRAP instructions save the PC of the packet after the TRAP call, in the exception link register (ELR)

Qualcomm
Confidential - May Contain Trade Secrets
2026-05-12 06:30:20 GMT
hyunchul.won@lge.com

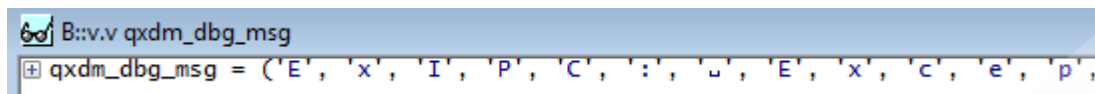
TCB Exception Codes

Event Num	Cause Code	Event Type	Event Description	Notes
0	0x0	Reset	Software thread reset	Non-maskable, Highest Priority
1	0x43	Imprecise	NMI	Non-maskable
1	0x42	Imprecise	Data abort	Non-maskable
1	0x44	Imprecise	Multiple TLB match	Non-maskable
1	0x45	Imprecise	Livelock exception	Non-maskable
2	0x01	Precise, Unrecoverable	Unrecoverable BIU error (bus error, timeout, L2 parity error, etc.)	Non-maskable
2	0x03	Precise, Unrecoverable	Double exception (exception occurs while SSR[EX]=1)	Non-maskable
2	0x11	Precise	Privilege violation: User/Guest mode execute to page with no execute permissions (X=0)	Non-maskable
2	0x12	Precise	Privilege violation: User mode execute to a page with no user permissions (X=1, U=0)	Non-maskable
2	0x15	Precise	Invalid packet.	Non-maskable
2	0x1A	Precise	Privilege violation: Executing a guest mode instruction in user mode	Non-maskable
2	0x1B	Precise	Privilege violation: Executing a supervisor instruction in user/guest mode	Non-maskable
2	0x1D	Precise, Unrecoverable	Packet with multiple writes to the same destination register	Non-maskable
2	0x1E	Precise, Unrecoverable	Program counter values that are not properly aligned.	Non-maskable
2	0x20	Precise	Load to misaligned address	Non-maskable
2	0x21	Precise	Store to misaligned address	Non-maskable
2	0x22	Precise	Privilege violation: User/Guest mode Read to page with no read permission (R=0)	Non-maskable
2	0x23	Precise	Privilege violation: User/Guest mode Write to page with no write permissions (W=0)	Non-maskable
2	0x24	Precise	Privilege violation: User mode Read to page with no user permission (R=1, U=0)	Non-maskable

Event Num	Cause Code	Event Type	Event Description	Notes
2	0x25	Precise	Privilege violation: User mode Write to page with no user permissions (W=1, U=0)	Non-maskable
3		Reserved		
4	0x60	TLB miss-X	Due to missing Fetch address on PC-page	Non-maskable
4	0x61	TLB miss-X	Due to missing Fetch on second page from packet that spans pages	Non-maskable
4	0x62	TLB miss-X	Due to icinva	Non-maskable
5		Reserved		
6	0x70	TLB miss-RW	Due to memory read	Non-maskable
6	0x71	TLB miss-RW	Due to memory write	Non-maskable
7		Reserved		
8	#u8	Trap0	Software Trap0 instruction	Non-maskable
9	#u8	Trap1	Software Trap1 instruction	Non-maskable
10		Reserved		
11	0xBF	Floating-Point	Execution of Floating-Point instruction resulted in exception	Non-maskable
12	0x80	Debug	Single-step debug exception	
13		Reserved		
14		Reserved		
15		Reserved		
16	0xC0	Interrupt0	General external interrupt	Maskable, highest priority general interrupt
17	0xC1	Interrupt1	General external interrupt	Maskable

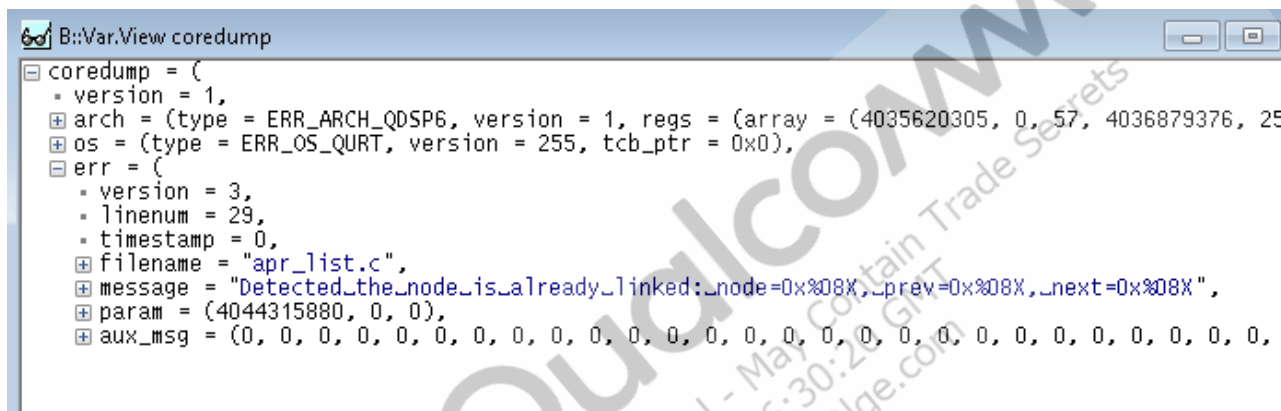
TRACE32 Commands

- `v.v qxdm_dbg_msg` shows thread name crashed



```
B:\v.v qxdm_dbg_msg
+ qxdm_dbg_msg = ('E', 'x', 'I', 'P', 'C', ':', 'u', 'E', 'x', 'c', 'e', 'p',
```

- `v.v coredump` provides more information on the error occurred



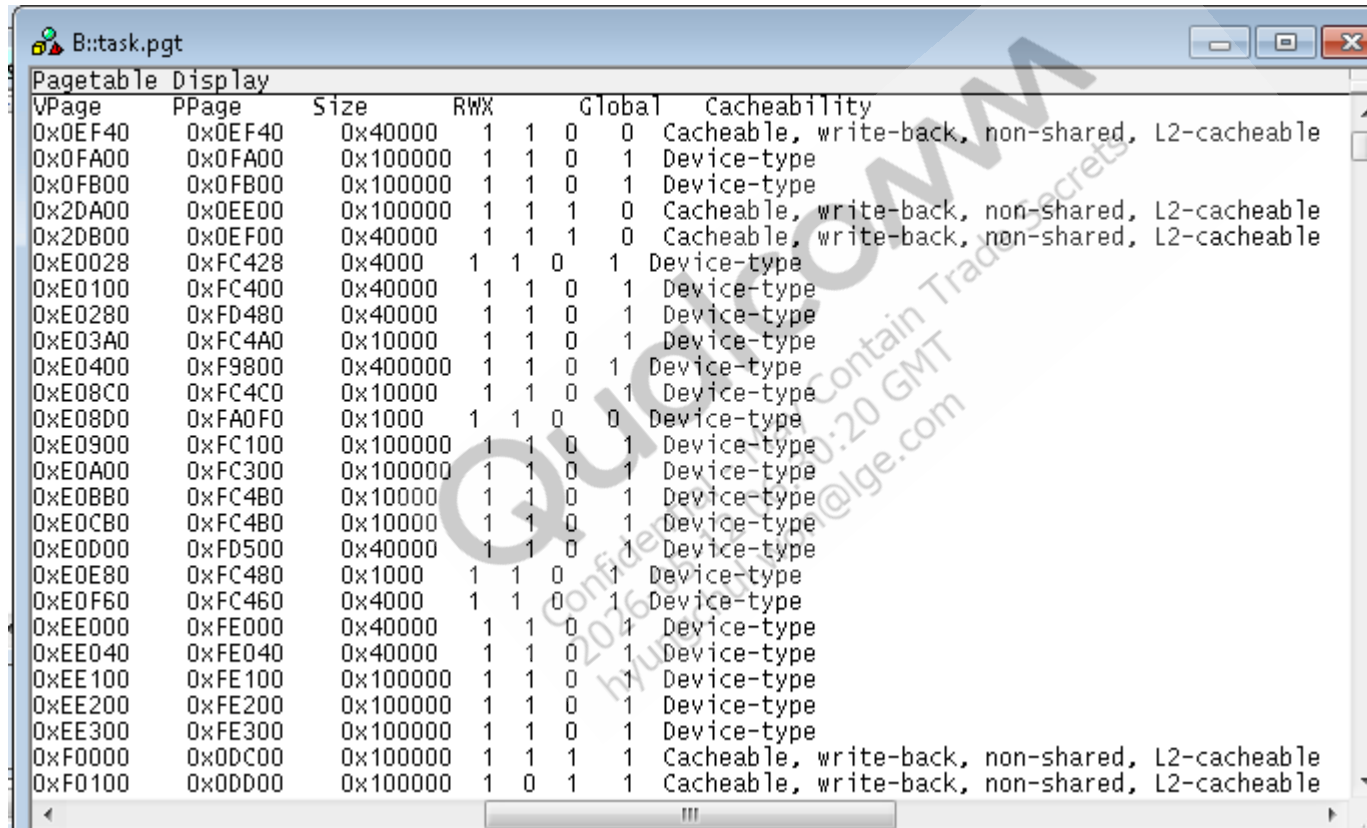
```
B:\Var.View coredump
+ coredump = (
  + version = 1,
  + arch = (type = ERR_ARCH_QDSP6, version = 1, regs = (array = (4035620305, 0, 57, 4036879376, 25
  + os = (type = ERR_OS_QURT, version = 255, tcb_ptr = 0x0),
  + err = (
    + version = 3,
    + linenum = 29,
    + timestamp = 0,
    + filename = "apr_list.c",
    + message = "Detected_the_node_is_already_linked:_node=0x%08X,_prev=0x%08X,_next=0x%08X",
    + param = (4044315880, 0, 0),
    + aux_msg = (0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
```

Useful TRACE32 commands:

`task.irql` → shows ISR lits
`task.pgt` → shows page table
`task.ftxlist` → shows futex list
`task.memlist` → shows memory details
`v.f` → shows current call stack
`mmu.list` → shows mmu details
`data.list <address>`

TRACE32 Commands (cont.)

- `task.pgt` is a useful command for understanding address mapping between physical and virtual memory mapping, as well as caching ability for the mapping



The screenshot shows a window titled "B::task.pgt" with a "Pagetable Display" header. The display contains a table with columns: VPage, PPage, Size, RWX, Global, and Cacheability. The table lists various memory mappings with their corresponding virtual and physical addresses, sizes, permissions, and cacheability attributes.

VPage	PPage	Size	RWX	Global	Cacheability
0x0EF40	0x0EF40	0x40000	1 1 0	0	Cacheable, write-back, non-shared, L2-cacheable
0x0FA00	0x0FA00	0x100000	1 1 0	1	Device-type
0x0FB00	0x0FB00	0x100000	1 1 0	1	Device-type
0x2DA00	0x0EE00	0x100000	1 1 1	0	Cacheable, write-back, non-shared, L2-cacheable
0x2DB00	0x0EF00	0x40000	1 1 1	0	Cacheable, write-back, non-shared, L2-cacheable
0xE0028	0xFC428	0x4000	1 1 0	1	Device-type
0xE0100	0xFC400	0x40000	1 1 0	1	Device-type
0xE0280	0xFD480	0x40000	1 1 0	1	Device-type
0xE03A0	0xFC4A0	0x10000	1 1 0	1	Device-type
0xE0400	0xF9800	0x400000	1 1 0	1	Device-type
0xE08C0	0xFC4C0	0x10000	1 1 0	1	Device-type
0xE08D0	0xFA0F0	0x1000	1 1 0	0	Device-type
0xE0900	0xFC100	0x100000	1 1 0	1	Device-type
0xE0A00	0xFC300	0x100000	1 1 0	1	Device-type
0xE0BB0	0xFC4B0	0x10000	1 1 0	1	Device-type
0xE0CB0	0xFC4B0	0x10000	1 1 0	1	Device-type
0xE0D00	0xFD500	0x40000	1 1 0	1	Device-type
0xE0E80	0xFC480	0x1000	1 1 0	1	Device-type
0xE0F60	0xFC460	0x4000	1 1 0	1	Device-type
0xEE000	0xFE000	0x40000	1 1 0	1	Device-type
0xEE040	0xFE040	0x40000	1 1 0	1	Device-type
0xEE100	0xFE100	0x100000	1 1 0	1	Device-type
0xEE200	0xFE200	0x100000	1 1 0	1	Device-type
0xEE300	0xFE300	0x100000	1 1 0	1	Device-type
0xF0000	0x0DC00	0x100000	1 1 1	1	Cacheable, write-back, non-shared, L2-cacheable
0xF0100	0x0DD00	0x100000	1 0 1	1	Cacheable, write-back, non-shared, L2-cacheable



Section 6

Case Studies

Qualcomm
Confidential - May Contain Trade Secrets
2026-05-12 06:30:20 GMT
hyunchul.won@lge.com

Case studies

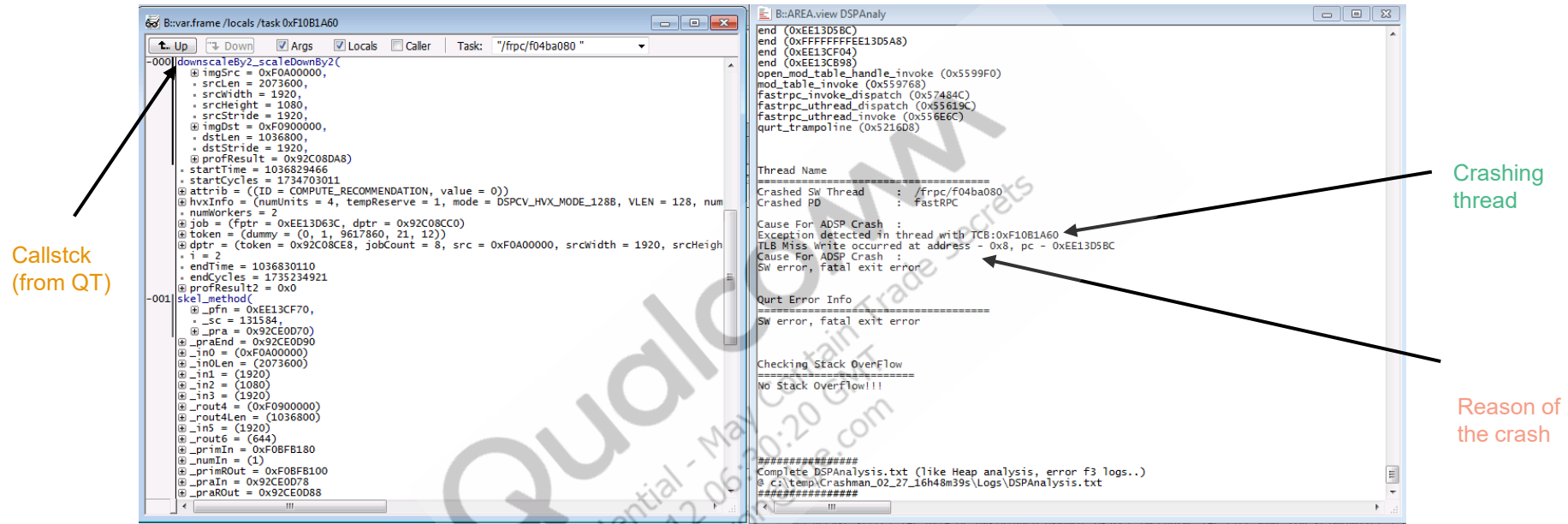
1. NULL pointer Access
2. Stack overflow
3. Mutex Deadlock
4. Heap Corruption
5. Loading Dynamic shared objects (.so's)

Qualcomm
Confidential - May Contain Trade Secrets
2026-05-12 06:30:20 GMT
hyungchul.won@lge.com

Case Study 1 – NULL Pointer Access

■ NULL Pointer Access Exception while running DownScaleBy2 example in Hexagon SDK

<SDK_root>\examples\common\downscaleBy2

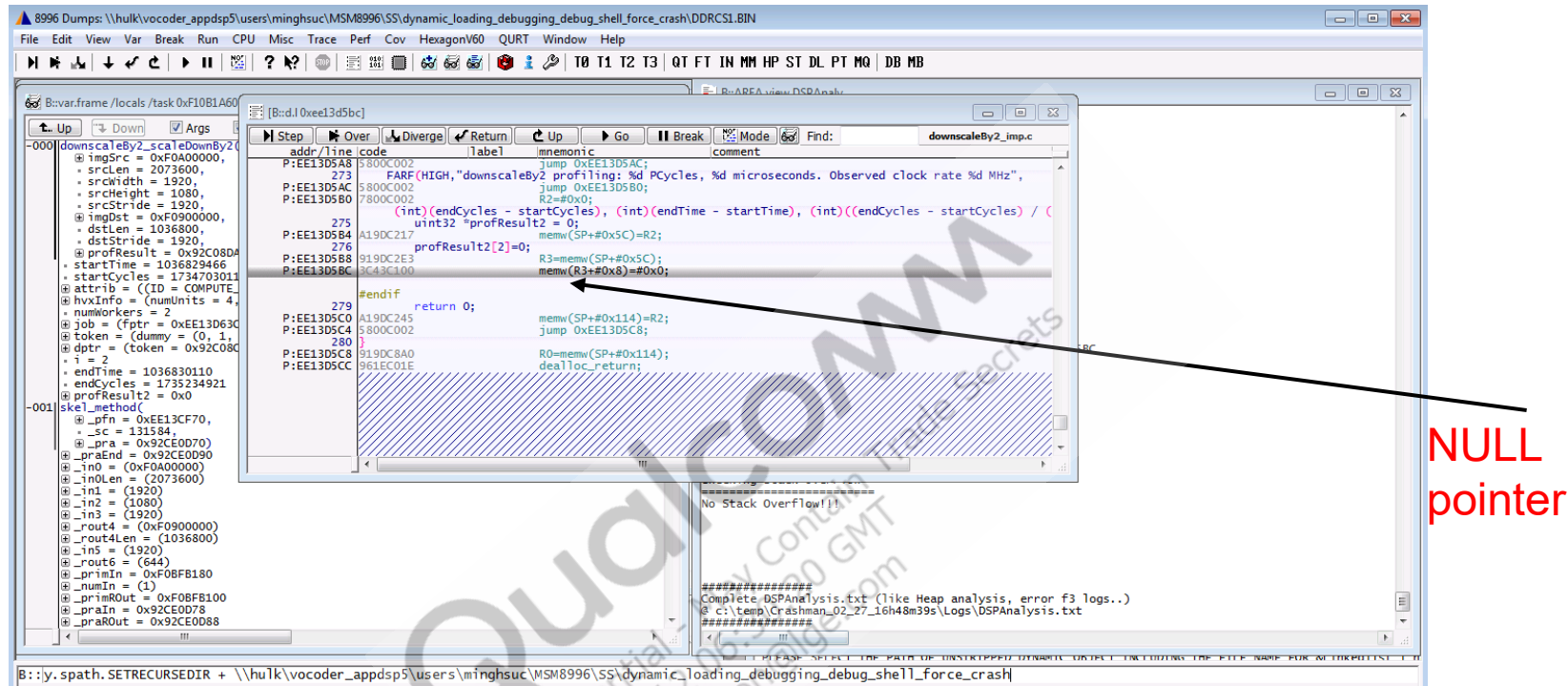


- Which thread it is? TCB: 0xF10B1A60
- What is the latest function in that thread? downscaleBy2_scaleDownBy2
- What is the immediate reason? Saving to address 0x8

■ Conclusion

- In function downscaleBy2_scaleDownBy2, there is a call saving to address 0x8

Case Study 1 – NULL Pointer Access (cont.)



- Set source to your own source folder as follows

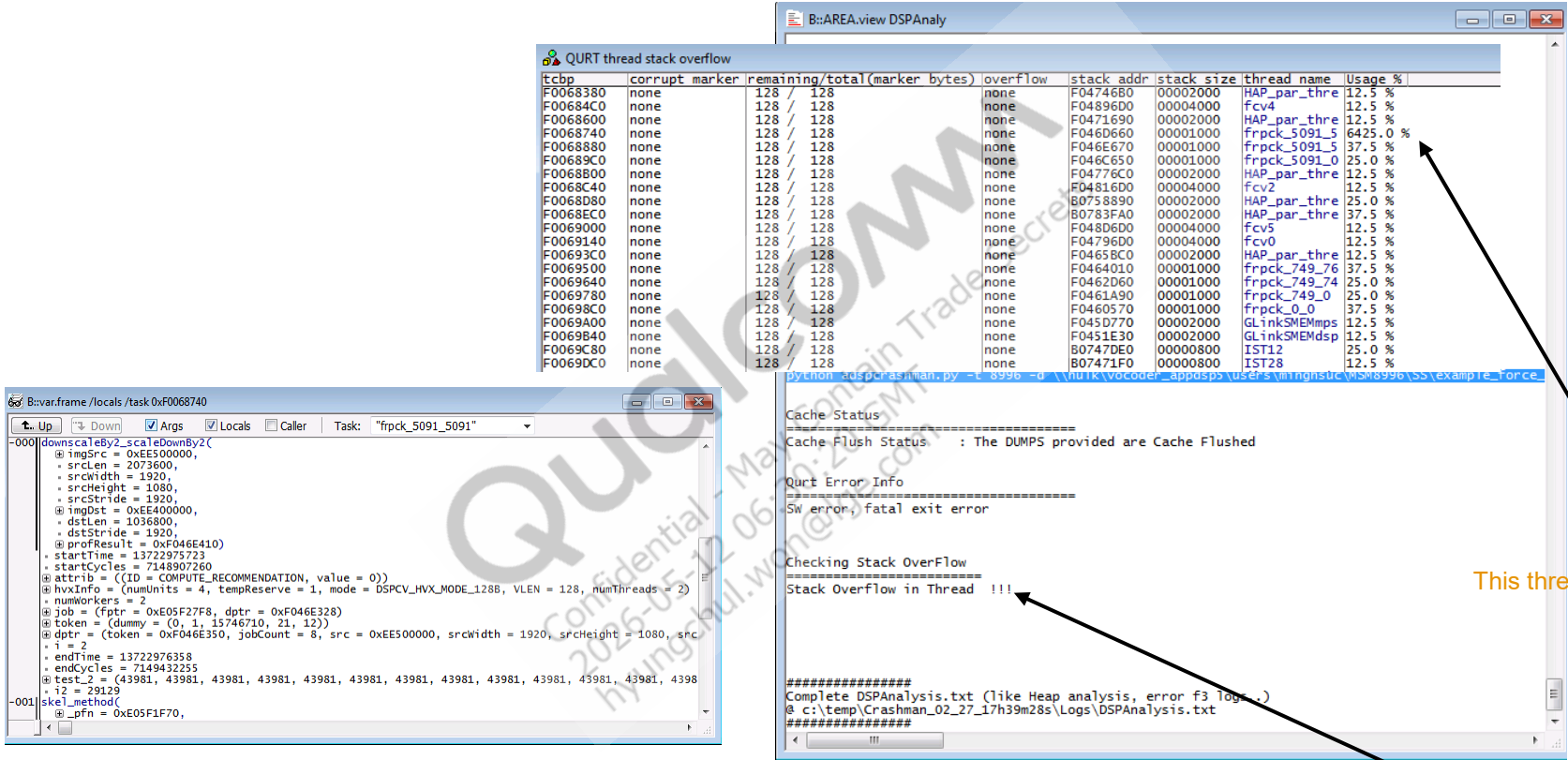
- cd adsp/cdsp build path
- y.SOURCEPATH.SETRECURSEDIR +

Conclusion

- profResult2 is NULL, and thus profResult[2] is invalid

Case Study 2 – Stack Overflow

- Stack Overflow in DownScaleBy2 Exception while running DownScaleBy2 example in Hexagon SDK
<SDK_root>\examples\common\downscaleBy2



- Conclusion
 - There is more space used than allocated in function downscaleBy2_scaleDownBy2

Stack overflow reported

Case Study 2 – Stack Overflow (cont.)

- In the code
 - test_2 has 64 K × size of (uint32_t) = 256 k
- Conclusion
 - Allocated stack size is less than 256 k
 - What is the allocated stack size then?
 - Roughly (used stack size)/(usage%) = (256 k / 6425%) = 4k in this case.

ize	thread name	Usage %
0	HAP_par_thre	12.5 %
0	fcv4	12.5 %
0	HAP_par_thre	12.5 %
0	frpck_5091_5	6425.0 %
0	frpck_5091_5	37.5 %
0	frpck_5091_0	25.0 %

```
[B:d:\downscaleBy2_scaleDownBy2]
Step Over Diverge Return Up Go Break Mode Find: downscaleBy2_imp.c
addr/line/source
248 // Stripe image to balance load across available threads, aiming for 3 stripes per
249 // worker, making sure rowsPerJob is multiple of 2
249 dptr.rowsPerJob = ((srcHeight + (3 * numWorkers - 1)) / (3 * numWorkers)) & ~1;
249 dptr.hvxInfo = &hvxInfo;
250 job.dptr = (void *)&dptr;
253 unsigned int i;
253 for (i = 0; i < numWorkers; i++)
256 {
256 // for multi-threaded impl, use this line.
256 (void) dspCV_worker_pool_submit(job);
256
256 // This line can be used instead of the above to directly invoke the
256 // callback function without dispatching to the worker pool. Useful
256 // to avoid multi-threading in debug scenarios to narrow down problems.
256 job.fptr(job.dptr);
262 }
263 dspCV_worker_pool_synctoken_wait(&token);
266 // clean up hvx configuration - release temporary reservation (if any), turn off power, etc.
266 dspCV_hvx_cleanup_mt_job(&hvxInfo);
271 // record end time (in both microseconds and pcycles) for profiling
271 #ifdef PROFILING_ON
271 // uint64 endCycles = HAP_perf_get_pcycles();
271 // uint64 endTime = HAP_perf_get_time_us();
272 uint64 endCycles = HAP_perf_get_pcycles();
273 *profResult = (uint32)(endTime - startTime);
274 FARF(HIGH, "downscaleBy2 profiling: %d PCycles, %d microseconds. Observed clock rate %d MHz",
274 (int)(endCycles - startCycles), (int)(endTime - startTime), (int)((endCycles - startCycles) / (
277 uint32 test_2[32768*2];
277 int i2=0;
278 while(1)
279 {test_2[i2++] = 0xABCD;}
```

Case Study 3 – Mutex Deadlock

```
Cache Status
=====
Cache Flush Status      : The DUMPS provided are Cache Flushed
CoreDump Info
=====
"PID 1 : "
Running CallStack
=====
-000|qurt_exception_raise_nonfatal()
-001|err_fatal_internal3()
-002|err_fatal_internal0()
-003|SNS_OS_MALLOC()
-003|SNS_OS_MALLOC()
-003|sns_sam_process_req()
-004|smr_csi_process_req_cb()
-005|qmi_csi_idl_err_translate()
-005|process_message()
-005|qmi_csi_xport_recv()
-006|dequeue_rx()
-006|xport_handle_event()
-007|qmi_csi_handle_event()
-008|sns_smr_task()
-009|qurt_trampoline()
----|end of frame

Stack Walker CallStack
=====
qurt_exception_raise_nonfatal (0x2DA25450)
err_fatal_internal2 (0x2DA3A5B0)
err_fatal_aux_msg (0x2DA3A5CC)
sns_sam_process_req (0x2DB58ECC)
smr_csi_process_req_cb (0x2DB5502C)
qmi_csi_xport_recv (0x2DB0F1E4)
xport_handle_event (0x2DB39A7C)
qmi_csi_handle_event (0x2DB0F9C0)
sns_smr_task (0x2DB54C64)
qurt_trampoline (0x2DB296E0)

Thread Name
=====
Crashed SW Thread      : SNS_SMR
Crashed PD             : Sensors

Cause For ADSP Crash :
Exception detected in thread with TCB:0xF0116480
Exit from user
Cause For ADSP Crash :
SW error, fatal exit error
```

**SNS_SMR is the thread
where Crashed happened.**

**SNS_SMR thread is running
in sensor PD**

Case Study 3 – Mutex Deadlock (cont.)

- From the call stack "SNS_SMGR" waits for "SNS_RH" and vice versa This leads to mutex deadlock situation.

Qurt Futex List

=====

Futex_addr	Object_type	Owner	Obj_Addr	Waiting_threads	
F000B850	mutex	SNS_SMGR	3459D960	SNS_RH	->NULL
F000B8C0	mutex	SNS_RH	2DA0FA50	SNS_SMGR	->NULL

- From call stack of "SNS_SMGR" and "SNS_RH" it is observed that both try to lock the same mutex and that leads to deadlock
- These threads do not release/free memory due to deadlock and leading to memory allocation failure (SNS_OS_MALLOC failed) and aDSP crashes:

From <crashman location>Logs\SensorF3Logs\f3lof.txt logs

```
01:23:01.625      762 amssheap.c    AMSS heap failed to allocate 56 bytes. Collecting heap statistics
01:23:01.625      384  sns_log_api_dsps_uimg.c  Low memory - Dropping 0 logpkts
01:23:01.625      777  amssheap.c    AMSS heap failed to allocate 56 bytes (failure 753)
01:23:01.625      193  qurt_elite_memory.cpp  malloc failed to allocate 56 bytes of memory requested by thread 0x44
in heap 0
01:23:01.625      243  sns_sam_service.c  SNS_OS_MALLOC failed - size 56
```

Case Study 4 – Heap Corruption

- Heap corruption messages similar to the example appears in DSPAnalysis.txt file:

Heap Corruption Detected

Current Pad: required Pad Value: 0xAAF2: 0xAAF9

Memory block got corrupted is 0xB07D5A60

Possible Overflown Block : 1425

Overflown Block's Header address : 0xB07D5A10

Size allocated to the Overflown Block: 80

<<It seems that the ptr 0xB07D5A10+0x10 passed in mem_free() is the culprit >>

This expected culprit(Overflown) block was allocated from function(that is, Owner of it) =
\\AUDIO_660\avtimer_server\avtimer_server_create_timer , task = 0x0 ,thread_id = 0x9

Case Study 4 – Heap Corruption (cont.)

- In TRACE32 using `d.dump 0xB07D5A60` command gets the address where the corruption happened

address	0	4	8	C	0123456789ABCDEF
D:B07D59F0	FF3ECDD7	FF22D28B	FF0B4ADE	FEF7455F	..>..."...J..._E..
D:B07D5A00	00000000	00000000	00000000	00000000	
D:B07D5A10	>4008AAF2	00000050	B030E9E0	00000009	...@P.....0.....
D:B07D5A20	DFDFDFDF	00000001	B0796EF8	80000000	ny.....
D:B07D5A30	B0796EF8	80000000	00000000	00000000	.ny.....
D:B07D5A40	0491A46F	00000000	0491EF39	00000000	o.....9.....
D:B07D5A50	00000000	00000000	00000000	00000000	
D:B07D5A60	0008AAF9	00000050	B030E9E0	00000009	P.....0.....

- Current pad is 0xAAF9 but required pad is 0xAAF2 (as mentioned in `DSPAnalysis.txt`). Due to which 0xB07D5A60 memory block is corrupted. So here the expected error is above the block that is 0xB07D5A10.

D:B07D5A70	CECECECE	00000001	B06EF808	80000000	n.....
D:B07D5A80	B06EF808	80000000	00000000	00000000	..n.....
D:B07D5A90	04506C7F	00000000	FFFFFFFF	0000FFFF	.lP.....
D:B07D5AA0	6AF7643C	00000001	00000000	00000000	<d.j.....
D:B07D5AB0	0008AAF2	00000050	B030E9E0	00000009	P.....0.....
D:B07D5AC0	CECECECE	00000001	B0796EF8	40000000	ny....@

Case Study 5 – Loading Dynamic Shared Objects

- See slide 21 for crashman command usage to generate crashman report with `dynamic.so`
- To debug crash/issue related to dynamic SO, in addition to RAM dumps and matching aDSP/cDSP elf, we need below files matching with the RAM dumps.
 - Matching `vmlinux` file
 - The `vmlinux` file extracts APPS page tables (SMMU mappings). As dynamic shared objects and their related content resides in APPS memory, the mappings must correlate with aDSP mappings and to view the contents. Mappings created by dynamic libraries in aDSP contain only Intermediate physical address (IPA) and not the actual Physical address (PA). Actual physical address is found in SMMU page tables.
 - Unstripped Dynamic object file
 - This file is needed to view the callstack or code flow related to dynamic object loaded in aDSP. This object file needs to be loaded at address that is dynamically allocated by `fastrpc`. This address is found out from structures of `fastrpc`. Crashman takes the SO files path as input and automatically loads corresponding SO at its address.

Case Study 5 – Loading Dynamic Shared Objects (cont.)

- Once dumps are loaded, from Crashman output folder `frpc_shell_names.txt`, we can see the dynamic objects loaded in aDSP. Example of an output is shown below:

```
/frpc/f04a2280
EntryPoint: 0xE0F00000
Shell Name: 0xF04A2308  "/frpc/f04a2280 calculato"
.So's Present in the Shell:
Start address: 0x1757E000, Obj Name:  "./libcalculator_skel.so"
```

In the above example, `/frpc/f04a2280` is the shell name created for dynamic object. `libcalculator_skel.so` is the SO file loaded at address `0x1757E000`.

- If SO file is not given as input to Crashman command, we can load it even after loading dumps.
- Use following command to load SO file at address defined in `frpc_shell_names.txt`.

```
d.load.elf libcalculator_skel.so 0x1757E000 /nocode /noclear)
```

- From Debug buffer (DB), we can see which dynamic process is created with the address space identifier (ASID).

```
Start user reaper thread for process 8
Entering main() ...
```

Case Study 5 – Loading Dynamic Shared Objects (cont.)

■ From QuRt Thread list, we can see corresponding threads related to this ASID

tcbptrr	UGP	threadID	priority	ASID	TNUM	state	Task_Name
F10B2400	2E50DF30	000010AD	C0	8		WAITING	/frpc/f04a2280
F10B2540	2E505A10	000020AE	C0	8		WAITING	HAP_par_thread_
F10B2680	2E5037C0	000020AF	C0	8		WAITING	HAP_par_thread_
F10B27C0	2E500010	000020B0	C0	8		WAITING	mem_gc_thread
F10B2900	F02B4350	000020B1	20	8		WAITING	user_reaper
F10B2B80	2E5002A0	000020B3	20	8		WAITING	/frpc/f04a2280
F10B3080	2E509CD0	000030B7	C0	8		WAITING	/frpc/f04a2280

■ Page tables in aDSP corresponding to this dynamic process is seen using command task.pgt 8

VPage	PPage	Size	RWX	Global_User	Cacheability	Raw
pagetable of asid 8						
0x171FC	0x980001	0x1000	RW - - U		Cacheable, write-back, non-shared, L2-cacheable	A08171FC 77300003
0x171FD	0x9800B9	0x1000	RW - - U		Cacheable, write-back, non-shared, L2-cacheable	A08171FD 77300173
0x1757E	0x9801A8	0x1000	R - X - U		Cacheable, write-back, non-shared, L2-cacheable	A081757E B7300351
0x1757F	0x9801A9	0x1000	RW - - U		Cacheable, write-back, non-shared, L2-cacheable	A081757F 77300353
.....						

Case Study 5 – Loading Dynamic Shared Objects (cont.)

- Since SO is loaded already, we can see the callstack of crashed thread `/frpc/f04a2280` by using command `v.f.`. Crashman output file `DSPAnalysis.txt` file contains information related to crash.
- If SO is not loaded, then the callstack does not show function names, instead it shows only from `DSPAnalysis.txt` file:

Cause For aDSP Crash:

Exception detected in thread with TCB: `0xF10B3080`

Store to misaligned address - `0x1`

Running CallStack

=====

-000|calculator_sum() → crash happened in calculator_sum api

-001|skel_method()

-002|P:0x1757E620()

-003|open_mod_table_handle_invoke()

-003|mod_table_invoke()

-004|fastrpc_invoke_dispatch()

-005|fastrpc_uthread_dispatch()

-006|fastrpc_uthread_invoke()

-007|qurt_trampoline()

----|end of frame

Case Study 5 – Loading Dynamic Shared Objects (cont.)

- To enable SMMU page tables as part of dumps, you need the following debug configurations enabled on the APPS side. By default these are disabled in the customer builds.

```
CONFIG_IOMMU_DEBUG  
CONFIG_IOMMU_DEBUG_TRACKING
```

Qualcomm
Confidential - May Contain Trade Secrets
2026-05-12 06:30:20 GMT
hyungchul.won@lge.com



Section 7

References

Qualcomm
Confidential - May Contain Trade Secrets
2026-05-12 06:30:20 GMT
hyunchul.won@lge.com

Documents and Standards

Title	Number
Qualcomm Technologies, Inc.	
<i>Qualcomm Hexagon V60/V61 Programmer's Reference Manual</i>	80-N2040-33
<i>Hexagon V60 Architecture System-Level Specification</i>	80-V9418-20
<i>Hexagon V60 HVX Programmer's Reference Manual</i>	80-N2040-30
<i>Hexagon Programming: Assembly Programming</i>	80-VB419-45

Qualcomm
Confidential - May Contain Trade Secrets
2026-05-12 06:30:20 GMT
hyungchul.won@lge.com

Acronyms and Terms

Acronym	Definition
aDSP	Audio digital signal processor
ADB	Android debug bridge
AMSS	Advanced mobile subscriber software
APPS	Application processor subsystem
ASID	Address space identifier
BADVA	Bad virtual address register
CDSP	Compute digital signal processor
COM	Communication port
DB	Debug buffer
DSP	Digital signal processor
ELF	Executable and linkable format
ELR	Exception link register
ETM	Embedded trace macrocell
FASTRPC	Fast remote procedure call
FT	Futex table
HVX	Hexagon vector extensions
IPA	Intermediate physical address
ISDB	Instruction set debug breakpoint
LPASS	Low power audio subsystem
MB	Merge buffer
MM	Memory manager

Acronyms and Terms

Acronym	Definition
MMU	Memory management unit
NMI	Non-maskable interrupt
OEM	Original equipment manufacturer
PA	Physical address
PD	Protection domain
QPST	Qualcomm product support tools
QPM	Qualcomm package manager
QTI	Qualcomm Technologies, Inc.
QuRT	Qualcomm real-time operating system
RAM	Random access memory
RTOS	Real-time operating system
SDK	Software development kit
SMMU	System memory management unit
SNS	Sensors subsystem
SO	Shared object
SSR	Subsystem restart
TCB	Thread control block
TLB	Translation lookaside buffer
TRACE32	Lauterbach TRACE32 debugger
vmlinux	Uncompressed Linux kernel image
xPCAT	Extended platform configuration analysis tool

Questions?

For additional information or to submit technical questions, go to: <https://www.qualcomm.com/support>

LEGAL INFORMATION

Your access to and use of this material, along with any documents, software, specifications, reference board files, drawings, diagnostics and other information contained herein (collectively this “Material”), is subject to your (including the corporation or other legal entity you represent, collectively “You” or “Your”) acceptance of the terms and conditions (“Terms of Use”) set forth below. If You do not agree to these Terms of Use, you may not use this Material and shall immediately destroy any copy thereof.

1) Legal Notice.

This Material is being made available to You solely for Your internal use with those products and service offerings of Qualcomm Technologies, Inc. (“**Qualcomm Technologies**”), its affiliates and/or licensors described in this Material, and shall not be used for any other purposes. If this Material is marked as “**Qualcomm Internal Use Only**”, no license is granted to You herein, and You must immediately (a) destroy or return this Material to Qualcomm Technologies, and (b) report Your receipt of this Material to qualcomm.support@qti.qualcomm.com. This Material may not be altered, edited, or modified in any way without Qualcomm Technologies’ prior written approval, nor may it be used for any machine learning or artificial intelligence development purpose which results, whether directly or indirectly, in the creation or development of an automated device, program, tool, algorithm, process, methodology, product and/or other output. Unauthorized use or disclosure of this Material or the information contained herein is strictly prohibited, and You agree to indemnify Qualcomm Technologies, its affiliates and licensors for any damages or losses suffered by Qualcomm Technologies, its affiliates and/or licensors for any such unauthorized uses or disclosures of this Material, in whole or part.

Qualcomm Technologies, its affiliates and/or licensors retain all rights and ownership in and to this Material. No license to any trademark, patent, copyright, mask work protection right or any other intellectual property right is either granted or implied by this Material or any information disclosed herein, including, but not limited to, any license to make, use, import or sell any product, service or technology offering embodying any of the information in this Material.

THIS MATERIAL IS BEING PROVIDED “AS IS” WITHOUT WARRANTY OF ANY KIND, WHETHER EXPRESSED, IMPLIED, STATUTORY OR OTHERWISE. TO THE MAXIMUM EXTENT PERMITTED BY LAW, QUALCOMM TECHNOLOGIES, ITS AFFILIATES AND/OR LICENSORS SPECIFICALLY DISCLAIM ALL WARRANTIES OF TITLE, MERCHANTABILITY, NON-INFRINGEMENT, FITNESS FOR A PARTICULAR PURPOSE, SATISFACTORY QUALITY, COMPLETENESS OR ACCURACY, AND ALL WARRANTIES ARISING OUT OF TRADE USAGE OR OUT OF A COURSE OF DEALING OR COURSE OF PERFORMANCE. MOREOVER, NEITHER QUALCOMM TECHNOLOGIES, NOR ANY OF ITS AFFILIATES AND/OR LICENSORS, SHALL BE LIABLE TO YOU OR ANY THIRD PARTY FOR ANY EXPENSES, LOSSES, USE, OR ACTIONS HOWSOEVER INCURRED OR UNDERTAKEN BY YOU IN RELIANCE ON THIS MATERIAL.

Certain product kits, tools and other items referenced in this Material may require You to accept additional terms and conditions before accessing or using those items.

Technical data specified in this Material may be subject to U.S. and other applicable export control laws. Transmission contrary to U.S. and any other applicable law is strictly prohibited.

Nothing in this Material is an offer to sell any of the components or devices referenced herein.

This Material is subject to change without further notification.

In the event of a conflict between these Terms of Use and the *Website Terms of Use* on www.qualcomm.com, the *Qualcomm Privacy Policy* referenced on www.qualcomm.com, or other legal statements or notices found on prior pages of the Material, these Terms of Use will control. In the event of a conflict between these Terms of Use and any other agreement (written or click-through, including, without limitation any non-disclosure agreement) executed by You and Qualcomm Technologies or a Qualcomm Technologies affiliate and/or licensor with respect to Your access to and use of this Material, the other agreement will control.

These Terms of Use shall be governed by and construed and enforced in accordance with the laws of the State of California, excluding the U.N. Convention on International Sale of Goods, without regard to conflict of laws principles. Any dispute, claim or controversy arising out of or relating to these Terms of Use, or the breach or validity hereof, shall be adjudicated only by a court of competent jurisdiction in the county of San Diego, State of California, and You hereby consent to the personal jurisdiction of such courts for that purpose.

2) Trademark and Product Attribution Statements.

Qualcomm is a trademark or registered trademark of Qualcomm Incorporated. Arm is a registered trademark of Arm Limited (or its subsidiaries) in the U.S. and/or elsewhere. The Bluetooth® word mark is a registered trademark owned by Bluetooth SIG, Inc. Other product and brand names referenced in this Material may be trademarks or registered trademarks of their respective owners.

Snapdragon and Qualcomm branded products referenced in this Material are products of Qualcomm Technologies, Inc. and/or its subsidiaries. Qualcomm patented technologies are licensed by Qualcomm Incorporated.

THE DOCUMENTATION ACCOMPANYING THE MATERIALS AND/OR RELEVANT PRODUCTS AND SERVICE OFFERINGS MAY INCLUDE IMPORTANT USE LIMITATIONS. ANY DEVIATIONS FROM APPLICABLE USE LIMITATIONS MAY ADVERSELY IMPACT PERFORMANCE, DURABILITY, QUALITY OR SAFETY. YOU ASSUME ALL RISKS AND LIABILITIES ASSOCIATED WITH ANY DEVIATIONS.